Universidad de Chile – Departamento de Ciencias de la Computación – CC5212

Guía Directa de Ejecución en el Cluster (`uhadoop`) y División de Trabajo (Grupo 13)

Continuación del Trabajo Previo de Preparación de Datos hacia Apache Pig y Apache Giraph

1. Punto de Partida (Trabajo Previo) y Ajustes según Feedback del Profesor

En el trabajo previo de preparación de datos (Colab del Grupo 13) ya se procesaron los 5,141,167 registros de ChileCompra 2025, corrigiendo las distorsiones de moneda (45 órdenes corregidas y 21 anomalías descartadas $\geq$ \$10,000M CLP) y generando los archivos base en Google Drive:

- `ordenes_limpias.tsv.bz2` (1,856,046 órdenes únicas, 30 columnas, columna `usar == 1` para las 1,853,398 válidas) y `ordenes_muestra.tsv` (10,000 filas).
- `nodos.tsv` (76,768 nodos con IDs disjuntos: `1XXXXXX` organismos y `2XXXXXX` proveedores) y `aristas.tsv` (476,709 aristas con `origen`, `destino`, `n_ordenes`, `monto_total_clp` y `log_monto`).

Ajustes técnicos para cumplir con lo solicitado por el Prof. Alberto Moya:

1. **Grafo Bipartito Bidireccional ($O \leftrightarrow P$) en Formato JSON de Lab 8:** En el trabajo previo, `aristas.tsv` solo tiene aristas dirigidas $O \rightarrow P$. En un grafo dirigido puro, los 1,165 organismos tienen grado de entrada cero ($\deg^-(u) = 0$) y quedan todos empatados con el PageRank mínimo ($1,95 \times 10^{-6}$). Al duplicar cada arista en ambos sentidos ($O \leftrightarrow P$) y convertirla al formato JSON del Laboratorio 8, la importancia fluye entre organismos y proveedores.
2. **PageRank Ponderado ($w_{uv} = \ln(1 + M_{uv})$) y los 2 Perfiles del Profesor:** Además del PageRank estándar del Lab 8, ejecutaremos una clase Java con pesos logarítmicos para cruzar el ranking financiero (Pig) con el estructural (Giraph) e identificar:
   - **Perfil 1 (Alto Monto y Bajo PageRank):** Proveedores con contratos millonarios concentrados en un solo organismo.
   - **Perfil 2 (Bajo/Moderado Monto y Alto PageRank):** Proveedores que abastecen transversalmente a los organismos más críticos del Estado.

2. División del Trabajo entre 3 Integrantes

Los Integrantes 1 y 2 trabajan **en paralelo** sobre el cluster `uhadoop.dcc.uchile.cl` sin bloquearse mutuamente:

Integrante	Módulo Responsable	Tareas Concretas	Entregables
Integrante 1 *Apache Pig y HDFS*	Rankings financieros, de volumen y agregaciones sectoriales en **Apache Pig**.	1. Subir `ordenes_muestra.tsv` y `ordenes_limpias.tsv.bz2` a HDFS. 2. Ejecutar `rankings_g13.pig` en muestra y en el dataset completo. 3. Medir tiempos con `time` y descargar los rankings con `-getmerge`.	• `ranking_por_monto.tsv` • `ranking_por_ordenes.tsv` • `ranking_por_organismos.tsv` • Tiempos de ejecución de Pig.
Integrante 2 *Apache Giraph*	Conversión del grafo al formato Lab 8 y ejecución de **PageRank en Giraph**.	1. Convertir `aristas.tsv` a JSON dirigido y bidireccional. 2. Ejecutar PageRank estándar (`SimplePageRankComputation`) y PageRank Ponderado (`WeightedPageRankJsonComputation`). 3. Medir tiempos de los 30 *supersteps* y descargar resultados.	• `pr_dirigido.tsv` • `pr_bidir_estandar.tsv` • `pr_bidir_ponderado.tsv` • Tiempos y logs de Giraph.
Integrante 3 *Cruce de Perfiles e Informe Final*	Cruce relacional (Pig + Giraph + `nodos.tsv`), análisis de perfiles y redacción final.	1. Cruzar los rankings del Integrante 1 y los PageRank del Integrante 2 usando `nodos.tsv`. 2. Extraer tablas y gráficos de los **Perfiles 1 y 2 del profesor**. 3. Consolidar el informe final y la presentación con métricas y tiempos.	• Tablas de Perfiles 1 y 2. • Gráficos comparativos. • Informe PDF y presentación final del grupo.

3. Integrante 1: Ejecución de Rankings en Apache Pig

3.1. 1. Crear el script `rankings_g13.pig` (compatible con las 30 columnas del trabajo previo)

Guarda el siguiente archivo como `rankings_g13.pig`. El filtro `usar == 1` descarta automáticamente la fila de cabecera y las 2,648 órdenes excluidas en el trabajo previo:

```
-- Archivo: rankings_g13.pig
raw_orders = LOAD '$INPUT_ORDENES' USING PigStorage('\t') AS (
    codigo:chararray, mes:int, fecha_envio:chararray, fecha_creacion:chararray,
    fecha_aceptacion:chararray, estado:chararray, estado_proveedor:chararray,
    tipo:chararray, procedencia:chararray, cod_licitacion:chararray,
    cod_convenio_marco:chararray, moneda:chararray, monto_original:double,
    monto_clp:double, monto_clp_corr:double, monto_corregido:int,
    monto_clp_limpio:double, flag_monto:chararray, n_items:int,
    cod_organismo:chararray, organismo:chararray, sector:chararray,
    region_comprador:chararray, proveedor_id:chararray, rut_proveedor:chararray,
    nombre_proveedor:chararray, es_persona_natural:chararray,
    region_proveedor:chararray, actividad_proveedor:chararray, usar:int
);

valid_orders = FILTER raw_orders BY usar == 1;

by_provider = GROUP valid_orders BY (proveedor_id, rut_proveedor, nombre_proveedor, es_persona_natural);

provider_metrics = FOREACH by_provider {
    orgs_unicos = DISTINCT valid_orders.cod_organismo;
    GENERATE
        FLATTEN(group) AS (proveedor_id, rut_proveedor, nombre_proveedor, es_persona_natural),
        SUM(valid_orders.monto_clp_limpio) AS monto_total_clp,
        COUNT(valid_orders) AS total_ordenes,
        COUNT(orgs_unicos) AS total_organismos_distintos;
};

rank_by_monto = ORDER provider_metrics BY monto_total_clp DESC;
STORE rank_by_monto INTO '$OUTPUT_DIR/ranking_por_monto' USING PigStorage('\t');

rank_by_ordenes = ORDER provider_metrics BY total_ordenes DESC;
STORE rank_by_ordenes INTO '$OUTPUT_DIR/ranking_por_ordenes' USING PigStorage('\t');

rank_by_organismos = ORDER provider_metrics BY total_organismos_distintos DESC;
STORE rank_by_organismos INTO '$OUTPUT_DIR/ranking_por_organismos' USING PigStorage('\t');
```

3.2. 2. Subir datos a HDFS, ejecutar Pig y descargar resultados

```
# 1. Subir archivos al servidor uhadoop (desde tu terminal)
scp ordenes_muestra.tsv ordenes_limpias.tsv.bz2 rankings_g13.pig usuario@uhadoop.dcc.uchile.cl:~/

# 2. Conectarse a uhadoop y cargar datos en HDFS (Pig lee .bz2 directamente sin descomprimir)
ssh usuario@uhadoop.dcc.uchile.cl
hdfs dfs -mkdir -p /uhadoop/grupo13/input
hdfs dfs -put ~/ordenes_muestra.tsv /uhadoop/grupo13/input/
hdfs dfs -put ~/ordenes_limpias.tsv.bz2 /uhadoop/grupo13/input/

# 3. Probar primero con la muestra (10.000 filas) y luego correr sobre el dataset completo
time pig -x mapreduce -param INPUT_ORDENES=/uhadoop/grupo13/input/ordenes_muestra.tsv \
    -param OUTPUT_DIR=/uhadoop/grupo13/out_pig_muestra ~/rankings_g13.pig

time pig -x mapreduce -param INPUT_ORDENES=/uhadoop/grupo13/input/ordenes_limpias.tsv.bz2 \
    -param OUTPUT_DIR=/uhadoop/grupo13/out_pig_completo ~/rankings_g13.pig

# 4. Unir y descargar las salidas desde HDFS
mkdir -p ~/resultados_pig
hdfs dfs -getmerge /uhadoop/grupo13/out_pig_completo/ranking_por_monto ~/resultados_pig/ranking_por_monto.tsv
hdfs dfs -getmerge /uhadoop/grupo13/out_pig_completo/ranking_por_ordenes ~/resultados_pig/ranking_por_ordenes.tsv
hdfs dfs -getmerge /uhadoop/grupo13/out_pig_completo/ranking_por_organismos ~/resultados_pig/
    ranking_por_organismos.tsv
```

4. Integrante 2: Ejecución de PageRank en Apache Giraph

4.1. 1. Transformar `aristas.tsv` al formato JSON del Laboratorio 8

Giraph en el Laboratorio 8 usa `JsonLongDoubleFloatDoubleVertexInputFormat`, donde cada línea es `[id_vertice, valor_inicial, [[id_vecino, peso], ...]]`. Ejecuta este script Python (`preparar_giraph.py`) sobre `aristas.tsv` para generar tanto el grafo dirigido ($O \rightarrow P$) como el bidireccional ($O \leftrightarrow P$) con peso $w_{uv} = \ln(1 + M_{uv})$:

```
# Archivo: preparar_giraph.py
import json
from collections import defaultdict

adj_dir, adj_bidir, nodes = defaultdict(list), defaultdict(list), set()

with open("aristas.tsv", "r", encoding="utf-8") as f:
```

```
    for line in f:
        p = line.strip().split("\t")
        if len(p) < 5 or not p[0].isdigit():
            continue
        u, v, w_log = int(p[0]), int(p[1]), round(float(p[4]), 6)
        nodes.update([u, v])
        adj_dir[u].append([v, w_log])
        adj_bidir[u].append([v, w_log])
        adj_bidir[v].append([u, w_log])

init_val = round(1.0 / len(nodes), 10)
with open("giraph_dirigido.txt", "w") as fa, open("giraph_bidireccional.txt", "w") as fb:
    for n in sorted(nodes):
        fa.write(json.dumps([n, init_val, adj_dir.get(n, [])]) + "\n")
        fb.write(json.dumps([n, init_val, adj_bidir.get(n, [])]) + "\n")
```

4.2. 2. Clase Java para el PageRank Ponderado (`WeightedPageRankJsonComputation.java`)

Mientras que para el PageRank estándar sin pesos usamos directamente `SimplePageRankComputation` del Lab 8, para el PageRank Ponderado usamos esta clase que pondera cada mensaje por $w_{uv}/\sum_k w_{uk}$:

```
package cl.uchile.pmd;

import org.apache.giraph.graph.BasicComputation;
import org.apache.giraph.graph.Vertex;
import org.apache.giraph.edge.Edge;
import org.apache.hadoop.io.DoubleWritable;
import org.apache.hadoop.io.FloatWritable;
import org.apache.hadoop.io.LongWritable;
import java.io.IOException;

public class WeightedPageRankJsonComputation extends BasicComputation<
    LongWritable, DoubleWritable, FloatWritable, DoubleWritable> {

    public static final int MAX_SUPERSTEPS = 30;
    public static final double DAMPING = 0.85;

    @Override
    public void compute(Vertex<LongWritable, DoubleWritable, FloatWritable> vertex,
                        Iterable<DoubleWritable> messages) throws IOException {
        if (getSuperstep() >= 1) {
            double sum = 0.0;
            for (DoubleWritable msg : messages) sum += msg.get();
            double rank = ((1.0 - DAMPING) / getTotalNumVertices()) + (DAMPING * sum);
            vertex.setValue(new DoubleWritable(rank));
        }
        if (getSuperstep() < MAX_SUPERSTEPS) {
            double totalWeight = 0.0;
            for (Edge<LongWritable, FloatWritable> edge : vertex.getEdges())
                totalWeight += edge.getValue().get();
            if (totalWeight > 0.0) {
                double currentRank = vertex.getValue().get();
                for (Edge<LongWritable, FloatWritable> edge : vertex.getEdges()) {
                    double w = edge.getValue().get();
                    sendMessage(edge.getTargetVertexId(),
                                new DoubleWritable(currentRank * (w / totalWeight)));
                }
            }
        } else {
            vertex.voteToHalt();
        }
    }
}
```

4.3. 3. Comandos en `uhadoop` para ejecutar las 3 corridas de Giraph

```
# 1. Subir los grafos JSON y el archivo Java a uhadoop y pasarlos a HDFS
scp giraph_dirigido.txt giraph_bidireccional.txt WeightedPageRankJsonComputation.java \
    usuario@uhadoop.dcc.uchile.cl:~/
ssh usuario@uhadoop.dcc.uchile.cl
hdfs dfs -put ~/giraph_dirigido.txt ~/giraph_bidireccional.txt /uhadoop/grupo13/input/

# 2. Localizar el JAR de Giraph del Lab 8
export GIRAPH_JAR=$(find / -name "giraph-examples*with-dependencies*.jar" 2>/dev/null | head -n 1)

# 3. Corrida 1: PageRank Estándar sobre Grafo Dirigido (0 -> P)
time hadoop jar $GIRAPH_JAR org.apache.giraph.GiraphRunner \
    org.apache.giraph.examples.SimplePageRankComputation \
    -vif org.apache.giraph.io.formats.JsonLongDoubleFloatDoubleVertexInputFormat \
    -vip /uhadoop/grupo13/input/giraph_dirigido.txt \
    -vof org.apache.giraph.io.formats.IdWithValueTextOutputFormat \
    -op /uhadoop/grupo13/out_pr_dirigido -w 1 \
    -mc org.apache.giraph.examples.SimplePageRankComputation\$SimplePageRankMasterCompute
```

```
# 4. Corrida 2: PageRank Estándar sobre Grafo Bidireccional (0 <-> P sin pesos)
time hadoop jar $GIRAPH_JAR org.apache.giraph.GiraphRunner \
    org.apache.giraph.examples.SimplePageRankComputation \
    -vif org.apache.giraph.io.formats.JsonLongDoubleFloatDoubleVertexInputFormat \
    -vip /uhadoop/grupo13/input/giraph_bidireccional.txt \
    -vof org.apache.giraph.io.formats.IdWithValueTextOutputFormat \
    -op /uhadoop/grupo13/out_pr_bidir_estandar -w 1 \
    -mc org.apache.giraph.examples.SimplePageRankComputation\$SimplePageRankMasterCompute

# 5. Compilar y ejecutar Corrida 3: PageRank Ponderado Bidireccional (0 <-> P con log_monto)
mkdir -p build
javac -cp "$(hadoop classpath):$GIRAPH_JAR" -d build WeightedPageRankJsonComputation.java
cp $GIRAPH_JAR giraph-weighted-g13.jar
jar uf giraph-weighted-g13.jar -C build .

time hadoop jar giraph-weighted-g13.jar org.apache.giraph.GiraphRunner \
    cl.uchile.pmd.WeightedPageRankJsonComputation \
    -vif org.apache.giraph.io.formats.JsonLongDoubleFloatDoubleVertexInputFormat \
    -vip /uhadoop/grupo13/input/giraph_bidireccional.txt \
    -vof org.apache.giraph.io.formats.IdWithValueTextOutputFormat \
    -op /uhadoop/grupo13/out_pr_bidir_ponderado -w 1

# 6. Descargar las 3 salidas de PageRank desde HDFS
mkdir -p ~/resultados_giraph
hdfs dfs -getmerge /uhadoop/grupo13/out_pr_dirigido ~/resultados_giraph/pr_dirigido.tsv
hdfs dfs -getmerge /uhadoop/grupo13/out_pr_bidir_estandar ~/resultados_giraph/pr_bidir_estandar.tsv
hdfs dfs -getmerge /uhadoop/grupo13/out_pr_bidir_ponderado ~/resultados_giraph/pr_bidir_ponderado.tsv
```

5. Integrante 3: Cruce de Resultados (Perfiles del Profesor) e Informe

Con los archivos descargados por el Integrante 1 (`ranking_por_monto.tsv`) y el Integrante 2 (`pr_bidir_ponderado.tsv`), más el diccionario `nodos.tsv` del trabajo previo, el Integrante 3 realiza el cruce final mediante el siguiente script (`cruzar_perfiles.py`):

```
# Archivo: cruzar_perfiles.py
# Uso: python3 cruzar_perfiles.py nodos.tsv ranking_por_monto.tsv pr_bidir_ponderado.tsv
import sys

nodos = {}
with open(sys.argv[1], encoding="utf-8") as f:
    for line in f:
        p = line.strip().split("\t")
        if len(p) >= 5 and p[0].isdigit() and p[1] == "PROVEEDOR":
            nodos[p[2]] = {"nid": int(p[0]), "rut": p[3], "nombre": p[4]}

pr = {}
with open(sys.argv[3], encoding="utf-8") as f:
    for line in f:
        p = line.strip().split("\t")
        if len(p) >= 2:
            pr[int(p[0])] = float(p[1])

datos = []
with open(sys.argv[2], encoding="utf-8") as f:
    for line in f:
        p = line.strip().split("\t")
        if len(p) >= 7 and p[0] in nodos:
            nid = nodos[p[0]]["nid"]
            datos.append({
                "codigo": p[0], "rut": p[1], "nombre": p[2],
                "monto": float(p[4]), "ordenes": int(p[5]), "orgs": int(p[6]),
                "pr": pr.get(nid, 0.0)
            })

datos.sort(key=lambda x: x["monto"], reverse=True)
for i, d in enumerate(datos, 1): d["rank_monto"] = i

datos.sort(key=lambda x: x["pr"], reverse=True)
for i, d in enumerate(datos, 1): d["rank_pr"] = i

perfil_1 = sorted([d for d in datos if d["rank_monto"] <= 50 and d["rank_pr"] > 500], key=lambda x: x["rank_monto"
    ])
perfil_2 = sorted([d for d in datos if d["rank_pr"] <= 30 and d["rank_monto"] > 150], key=lambda x: x["rank_pr"])

print("=== PERFIL 1: ALTO MONTO Y BAJO PAGERANK (CONTRATOS CONCENTRADOS) ===")
for d in perfil_1[:10]:
    print(f"Monto #{d['rank_monto']:<3} | PR #{d['rank_pr']:<6} | Orgs: {d['orgs']:<3} | ${d['monto']/1e6:,.1f}M |
        {d['nombre']}")

print("\n=== PERFIL 2: BAJO/MODERADO MONTO Y ALTO PAGERANK (PROVEEDORES TRANSVERSALES) ===")
for d in perfil_2[:10]:
    print(f"PR #{d['rank_pr']:<3} | Monto #{d['rank_monto']:<6} | Orgs: {d['orgs']:<3} | ${d['monto']/1e6:,.1f}M |
        {d['nombre']}")
```

Estructura Recomendada para el Informe Final y la Presentación

1. **Preparación y Limpieza de Datos:** Resumen del trabajo previo sobre los 5, 14M de registros, reglas de corrección de divisas (45 órdenes corregidas y 21 excluidas $\geq$ \$10,000M CLP) y reducción de \$45, 80B a \$18, 18B CLP.
2. **Arquitectura en el Cluster (`uhadoop`):** Diseño del pipeline dual (Apache Pig para agregaciones tabulares y Apache Giraph para centralidad en el grafo comprador–proveedor) y tiempos de ejecución registrados.
3. **Efecto de la Topología del Grafo en Giraph:** Comparación entre el grafo dirigido simple ($O \to P$) y el grafo bipartito bidireccional ponderado ($O \leftrightarrow P$ con $w_{uv} = \ln(1 + M_{uv})$), mostrando también los organismos públicos más centrales.
4. **Análisis de los 2 Perfiles del Profesor:** Discusión de casos reales del **Perfil 1** (ej. *Sonda S.A.*, *Construcciones y Prestaciones Petroleras S.A.*, *Constructora Digua*) frente al **Perfil 2** (ej. *PRISA S.A.*, *Carozzi S.A.*, *Blue Express S.A.*, *Dimerc S.A.*).